Memcached

Once you've configured Drupal to use **Memcached** as its default cache backend, any cache operations performed by your custom block will automatically use Memcached. You don't need to add any specific Memcached-related code to your block itself. Instead, you use Drupal's built-in **Cache API** to make your block's output cacheable.

The key is to properly set the **cacheability metadata** within your block's build() method. This metadata tells Drupal *what* to cache, *how long* to cache it, and *what* conditions should cause the cache to be invalidated.

1. The Core Concept: Cacheability Metadata

In Drupal, you define the caching behavior of a block by adding a #cache key to its render array. This key contains three main properties:

- max-age: How long the cache should be considered valid. Use Cache::PERMANENT to cache indefinitely or an integer (in seconds) for a temporary cache.
- tags: An array of **cache tags**. When any of these tags are invalidated, Drupal automatically clears the cached block output. This is the most powerful and precise way to invalidate the cache. For example, if your block displays a list of nodes, you would add the node_list tag to ensure the cache is cleared whenever a node is added, updated, or deleted.
- contexts: An array of **cache contexts**. These tell Drupal to create a different cache entry for each variation of a specific context. For example, using 'user.roles' will create a separate cache entry for each user role, ensuring different users see content relevant to their permissions.

2. Example: Caching a Custom Block

Let's assume you have a custom block in a module called mymodule. This block displays a list of the 5 most recently created articles.

```php
<?php
namespace Drupal\mymodule\Plugin\Block;

use Drupal\Core\Block\BlockBase;
use Drupal\Core\Cache\Cache;
use Drupal\Core\Plugin\ContainerFactoryPluginInterface;
use Symfony\Component\DependencyInjection\ContainerInterface;
use Drupal\node\Entity\Node;

/**
 * Provides a 'Recent Articles' block.
 *
 * @Block(
 * id = "mymodule_recent_articles",
 * admin_label = @Translation("Recent Articles"),
 * )
 */
class RecentArticlesBlock extends BlockBase {

  /**
   * {@inheritdoc}
   */
  public function build() {
    $nids = \Drupal::entityQuery('node')
      ->condition('status', 1)
      ->condition('type', 'article')
      ->sort('created', 'DESC')
      ->range(0, 5)
      ->execute();
```

```
    $nodes = Node::loadMultiple($nids);

    $items = [];
    foreach ($nodes as $node) {
      $items[] = $node->toLink($node->getTitle());
    }

    $build = [
      '#theme' => 'item_list',
      '#items' => $items,
      '#cache' => [
        // Cache the block indefinitely.
        'max-age' => Cache::PERMANENT,
        // Invalidate the cache whenever any node changes.
        'tags' => ['node_list'],
        // Create a different cache entry for each language.
        'contexts' => ['languages:language_interface'],
      ],
    ];

    return $build;
  }

}
```

3. How the Caching Works with Memcached

1. **Request:** A user requests a page containing your RecentArticlesBlock.
2. **Cache Check:** Drupal's render system looks for a cached version of the block.
3. **Cache Miss:** If no cache entry exists (or if it has been invalidated), Drupal executes the build() method to render the block.
4. **Database Query:** The block runs a database query to get the recent articles.
5. **Caching:** Before rendering the final page, Drupal sees the #cache metadata in the block's render array. It takes the rendered HTML output of the block, serializes it, and saves it to a cache bin.
6. **Memcached's Role:** Since you've configured Memcached as the default backend, Drupal doesn't save this cached data to the database. Instead, it sends the data to the Memcached server, which stores it in RAM for fast retrieval. The cache key will be a unique hash based on the block ID, cache contexts, and other factors.
7. **Subsequent Requests:** The next time a user requests the same page, Drupal will find the cached block output in Memcached and serve it directly, without ever executing the expensive database query in your build() method.
8. **Invalidation:** When a new article is created, Drupal automatically invalidates the node_list cache tag. The next time the page is requested, Drupal will recognize that the cache is stale and re-execute the build() method, generating a new, up-to-date block output that is then saved to Memcached.

Memcached Configure `settings.php`:

```
$settings['cache']['default'] = 'cache.backend.memcache';
$settings['memcache']['servers'] = ['127.0.0.1:11211' => 'default'];
$settings['memcache']['bins'] = ['default' => 'default'];
$settings['cache']['bins']['render'] = 'cache.backend.memcache';
$settings['cache']['bins']['dynamic_page_cache'] = 'cache.backend.memcache';
$settings['cache']['bins']['page'] = 'cache.backend.memcache';
```

🧩 2. Enable Caching in Your Custom Block

If you're writing a custom block plugin, make sure to return a **render array** with cache metadata.

✅ Example: Custom Block with Caching

```
namespace Drupal\my_module\Plugin\Block;
use Drupal\Core\Block\BlockBase;
/**
 * Provides a custom block with cache.
 *
 * @Block(
 *   id = "my_custom_cached_block",
 *   admin_label = @Translation("My Cached Block")
 * )
 */
class MyCachedBlock extends BlockBase {
  /**
   * {@inheritdoc}
   */
  public function build() {
    $build = [
      '#markup' => $this->t('This block is cached.'),
      '#cache' => [
        'contexts' => ['user.roles'],       // Varies by user role
        'tags' => ['my_block:data'],        // Used for invalidation
        'max-age' => 3600,                  // Cache lifetime in seconds
```

```
      ],
    ];
    return $build;
  }
}
```

✅ Because the `render` cache bin is now backed by Memcached, this block output will be cached there.

🛠️ 3. Invalidate or Clear Cache When Needed

To programmatically invalidate the block cache:

```
\Drupal::service('cache_tags.invalidator')->invalidateTags(['my_block:data']);
```

Or via Drush:

```
drush cache:rebuild
```

📦 Optional: Use Cache API Directly (for non-render data)

If you want to manually cache custom data:

```
use Drupal\Core\Cache\Cache;
$cid = 'my_module:some_key';
$cache = \Drupal::cache('default'); // Uses memcached backend
// Save data
$cache->set($cid, $data, Cache::PERMANENT, ['my_tag']);
// Get cached data
if ($cache_item = $cache->get($cid)) {
  $data = $cache_item->data;
}
// Invalidate
\Drupal::service('cache_tags.invalidator')->invalidateTags(['my_tag']);
```

What is the purpose of using services and dependency injection instead of global functions in Drupal?

In **Drupal**, using **services and dependency injection** instead of **global functions** is considered a best practice for several reasons—most importantly for **maintainability, testability, and modularity**.

Here's a breakdown of **why Drupal uses services and dependency injection** over global functions:

✅ 1. Testability

- **Global functions** are hard to mock or replace during unit testing.
- **Services**, when injected, can be **mocked or stubbed** easily.
- This enables **automated testing** of components in isolation, which is critical in modern development.

  Example: You can mock the `logger.factory` service in a test, but you can't easily mock a global `watchdog()` call.

✅ 2. Loose Coupling and Modularity

- Global functions create **tight coupling** between components.
- **Dependency Injection (DI)** allows components to depend only on what they need, making the system more modular.

  If a class uses `\Drupal::config()` directly, it is tightly coupled to the global Drupal system. With DI, the class just gets a `ConfigFactoryInterface`, making it reusable and decoupled from Drupal internals.

✅ 3. Code Reusability and Flexibility

- When you inject services, you can **swap out implementations** (e.g., use a different mailer service).
- This makes your code **more flexible** and customizable.

✅ 4. Better for Object-Oriented Programming (OOP)

- Modern Drupal is built on **Symfony components**, which use OOP.
- OOP encourages **encapsulation** and **inversion of control (IoC)**—both enabled by services and DI.

✅ 5. Performance (in some contexts)

- Services are **instantiated only when needed** (lazy-loaded).
- Global functions may load unnecessary dependencies or logic even when not needed.

✅ 6. Service Container and Central Configuration

- Services are defined in the **Drupal service container** (`services.yml`) and can be **overridden**, **extended**, or **decorated**.
- This provides centralized control over what classes are used in different parts of the system.

🚫 Problems with Global Functions

Global Function Issue	Why It's Bad
Hard to test	No easy way to replace in tests
Tightly coupled	Code depends on global state
Hidden dependencies	Hard to see what a class actually needs
Difficult to extend	Can't swap implementations easily

📌 Example: Injecting Logger vs Using Global Function

❌ Bad (Global Function):

```
\Drupal::logger('my_module')->notice('Something happened.');
```

✅ Good (Dependency Injection):

```
class MyService {
  protected $logger;
  public function __construct(LoggerChannelFactoryInterface $logger_factory) {
    $this->logger = $logger_factory->get('my_module');
  }
  public function doSomething() {
    $this->logger->notice('Something happened.');
  }
}
```

💡 Summary

Feature	Global Functions	Services + DI
Testable	❌	✅
Reusable	❌	✅
Extensible	❌	✅
Follows OOP	❌	✅
Follows Best Practices	❌	✅

🧠 TL;DR

Drupal uses **services and dependency injection** to promote **clean, testable, decoupled code**, which aligns with **modern PHP standards** and the **Symfony framework** it's built on. Global functions, while simpler, do not scale well in large or professional applications.

Let me know if you want examples of defining your own service or injecting one in a custom module.

The disadvantages of using global functions in Drupal, and the shift towards services and dependency injection, are rooted in modern object-oriented programming principles.

Disadvantages of Global Functions in Drupal 👹

Before Drupal 8, a lot of Drupal's core functionality relied on global functions like `node_load()` or `user_save()`. While they were easy to use, they had significant drawbacks:

- **Tight Coupling:** Global functions are accessible from anywhere in the codebase, which creates tight, hidden dependencies. A change to one function could unintentionally break another part of the system, making the code harder to maintain and less flexible.
- **Poor Testability:** It's difficult to create isolated unit tests for code that uses global functions. To test a single function, you might need to "bootstrap" the entire Drupal environment or set up a complex mock environment to mimic the global state, which is inefficient and brittle.
- **Namespace Pollution:** Global functions reside in the global namespace, increasing the risk of naming conflicts, especially when a project uses many custom modules or third-party libraries.
- **Lack of Encapsulation:** A global function's dependencies aren't explicitly stated. You have to read the function's source code to understand what other functions or data it relies on, which violates the principle of information hiding.
- **Hidden Dependencies and Side Effects:** It's not immediately clear what a global function does or what it affects. A function might modify global state or variables, leading to unexpected side effects that are difficult to track down and debug.

Why Services and Dependency Injection are Used in Drupal 🤝

In Drupal 8 and beyond, the framework shifted its architecture to embrace a more object-oriented approach using **services** and **dependency injection**, powered by the **Symfony framework**.

- **Services** are PHP classes that perform a specific, reusable task. Think of them as dedicated tools with a single responsibility, like an email sender, a database connection, or a logger. They are registered in a **service container**, which acts as a central registry for all available "tools" in the application. You define a service in a YAML file (`.services.yml`), which maps a unique machine name to a PHP class.
- **Dependency Injection (DI)** is a design pattern that provides a class with the objects it needs (its dependencies) instead of letting the class create them itself. Instead of a class calling a global function or fetching a service from the container directly, the services it needs are "injected" into it, typically through its constructor.

This approach provides several key advantages:

1. **Loose Coupling:** DI breaks the tight connections between components. A class doesn't need to know how its dependencies are created, only that it will receive them. This allows you to easily swap out one service for another without changing the code that uses it.
2. **Improved Testability:** Because a class's dependencies are explicitly passed in, you can easily **mock** them during unit testing. This lets you test a class in isolation without worrying about its real-world dependencies, making tests faster, more reliable, and easier to write.
3. **Encapsulation and Clarity:** The dependencies of a class are immediately obvious by looking at its constructor. This explicit declaration makes the code easier to read, understand, and debug. It adheres to the principle of "separation of concerns" where each component has a clear and defined role.
4. **Flexibility and Reusability:** Services are designed to be reusable and can be configured and overridden by other modules. This allows for a more flexible architecture where different modules can provide alternative implementations for the same service, such as a different caching or email system.

By moving from a procedural, global-function-based architecture to a modern, object-oriented one with services and DI, Drupal became a more robust, maintainable, and scalable framework. This change was

a fundamental part of a larger initiative to modernize the platform and align it with current best practices in PHP development.

Disadvantages of global function in Drupal and why services and dependency injection are used in Drupal

In Drupal (especially from Drupal 8 onward), **global functions** are generally discouraged in favor of **services** and **dependency injection (DI)**. Here's a breakdown of the **disadvantages of global functions** and why Drupal prefers using **services with dependency injection**.

🔻 Disadvantages of Global Functions in Drupal

1. ❌ **Tight Coupling**

   Global functions create strong dependencies between parts of the code, making it difficult to isolate and test individual components.

2. ❌ **Hard to Unit Test**
   - Global functions cannot be easily mocked or replaced during testing.
   - This makes unit testing challenging, especially when the function uses hardcoded logic or external systems (e.g., database access, file systems).

3. ❌ **No Configuration Flexibility**
   - Global functions cannot be easily configured or extended.
   - For example, you can't swap out part of the logic without rewriting the function itself or adding hacks.

4. ❌ **Hidden Dependencies**
   - When using a global function, it's not immediately clear what dependencies it relies on.
   - This violates the principle of **explicit dependencies**—where everything a component needs should be passed to it.

5. ❌ **Poor Reusability**
   - Global functions are often procedural and not reusable in object-oriented design.
   - They limit how modular and reusable your codebase can be.

6. ❌ **Namespace Pollution**
   - Global functions live in the global namespace, which increases the risk of naming collisions and makes code harder to organize.

✅ Why Drupal Uses Services and Dependency Injection

Drupal moved toward modern object-oriented practices starting in **Drupal 8**, adopting **Symfony components**. This brought in the **service container** and **dependency injection**. Here's why:

1. ✅ Loose Coupling

- Services are injected into classes, which reduces coupling.
- Each class only depends on the services it needs.

2. ✅ Better Testing

- Services can be mocked or replaced in tests.
- Easier to write unit and functional tests.

3. ✅ Configurability

- Services are defined in YAML and can be overridden or extended easily using service tags or service alteration hooks.

4. ✅ Encapsulation

- Services encapsulate specific functionality (e.g., email sending, logging, file storage).
- This leads to better organization and separation of concerns.

5. ✅ Explicit Dependencies

- Constructor injection makes it clear what a class depends on.
- Helps with code readability and maintenance.

6. ✅ Interchangeability

- Services can be replaced or decorated to modify behavior without changing core logic.
- Useful in contrib/custom modules.

📌 Example

❌ Global function usage (bad practice):

```
function my_module_do_something() {
  $node = \Drupal\node\Entity\Node::load(1);
  // ...
}
```

- Uses hard-coded static call.
- Difficult to test or override.

✅ **Using service and dependency injection (best practice):**

```
use Drupal\node\NodeStorageInterface;

class MyService {
  protected $nodeStorage;

  public function __construct(NodeStorageInterface $nodeStorage) {
    $this->nodeStorage = $nodeStorage;
  }

  public function doSomething() {
    $node = $this->nodeStorage->load(1);
    // ...
  }
}
```

- Uses constructor injection.
- Easy to mock `NodeStorageInterface` in tests.
- Decoupled and modular.

✅ **Summary**

Criteria	Global Functions ❌	Services & DI ✅
Testability	Poor	Excellent
Flexibility	Rigid	Configurable
Code Organization	Messy	Modular
Dependency Management	Hidden	Explicit
Extensibility	Limited	High

Let me know if you want a practical example using `*.services.yml` or how to define and inject your own custom service in Drupal.